

MLA: DeepSeek’s Low-Rank KV Compression, Worked Out by Hand

CACHE ONE SHARED LATENT, NOT FULL PER-HEAD K AND V — A 98% KV REDUCTION, BY HAND

What this document does. It works through Multi-head Latent Attention (MLA), the KV-compression scheme behind DeepSeek-V2/V3. Instead of caching full per-head K and V , MLA caches a single low-rank *latent* c_{KV} of dimension c_{dim} (plus a small decoupled-RoPE dimension), and reconstructs per-head K, V on the fly via learned up-projections. We derive the compression ratio $\text{ratio} = (c_{\text{dim}} + \text{rope}) / (2H_q d_{\text{head}})$ and compute it for DeepSeek-V3 ($c_{\text{dim}} = 512$, $\text{rope} = 64$, $H_q = 128$, $d = 128$): exactly $576/32768 = 0.0176$, a **98.2%** reduction versus full MHA (✓). We also show why the extra up-projection matmuls are nearly free thanks to the “absorbed matmul” trick. Every figure comes from Appendix A (`m03_t04_mla_engine.py`).

The one idea. GQA (Topic 3) shrinks the KV cache by *sharing* K/V heads. MLA shrinks it by *compressing*: project the full K/V information into a small shared latent, store that, and recover the per-head detail with cheap matmuls at attention time. The result is a far larger reduction than GQA — $\sim 57\times$ on a like-for-like layer count — with accuracy that matches or exceeds full MHA.

CONCEPTS COVERED, IN ORDER

the MLA forward pass (down-project to cache, up-project to reconstruct); the decoupled RoPE dimension; what is actually cached; the compression-ratio formula; the DeepSeek-V3 worked example (0.0176); the absorbed-matmul trick; and why MLA must be trained in, not retrofitted.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The idea: cache the latent, not the heads	2
2	Step 1 — The MLA forward pass	2
3	Step 2 — What is actually cached	2
4	Step 3 — The compression ratio	3
5	Step 4 — Why the up-projections are nearly free: the absorbed matmul	3
6	Step 5 — The catch: MLA must be trained in	4
7	The argument, at a glance	4
	7.1 Equation cheat-sheet	4
	Appendix A — Reference implementation	4

1 The idea: cache the latent, not the heads

GQA and MQA reduce the KV cache by reducing the *number* of K/V heads. MLA takes a different route: it keeps the full expressiveness of many heads but stores only a compressed *latent* from which the heads are reconstructed.

Intuition

Think of the per-token K and V across all 128 heads ($2 \times 128 \times 128 = 32768$ numbers) as a high-dimensional object that is actually low-rank — most of its information lives in a much smaller subspace. MLA learns a down-projection into that subspace (512 numbers, the latent c_{KV}), caches only the latent, and learns up-projections W^{UK}, W^{UV} that expand it back into per-head K, V when attention needs them. You pay a little compute to save a lot of memory — and decode is memory-bound, so that is the trade you want.

2 Step 1 — The MLA forward pass

For one new token with hidden state x :

MLA forward pass — one token

$$\begin{aligned}
 c_{KV} &= x W^{DKV} & [1 \times 512] & \leftarrow \text{this is what we cache} \\
 K &= c_{KV} W^{UK} & [1 \times H_q d] & \rightarrow \text{reshape to } [H_q \times d] \\
 V &= c_{KV} W^{UV} & [1 \times H_q d] & \rightarrow \text{reshape to } [H_q \times d] \\
 c_Q &= x W^{DQ} & [1 \times 1536] & \text{(query also compressed)} \\
 Q &= [c_Q W_{\text{nope}}^{UQ} ; (c_Q W_{\text{rot}}^{UQ}) \oplus \text{RoPE}]
 \end{aligned}$$

Down-projection W^{DKV} compresses x into the small latent c_{KV} ; only c_{KV} enters the cache. Up-projections W^{UK}, W^{UV} reconstruct per-head K, V *inside the attention kernel*, as needed. The query is split into a non-rotated part and a RoPE-rotated part — the *decoupled RoPE* — which is why the cache also carries a small extra rope dimension.

Watch out

RoPE cannot be applied to the compressed latent directly — rotating before up-projection does not commute with the per-head reconstruction. DeepSeek's fix is the *decoupled* RoPE: a small separate dimension (here 64) carries the rotary part, cached alongside the latent. That is why the cache size is $c_{\text{dim}} + \text{rope}$, not c_{dim} alone.

3 Step 2 — What is actually cached

The whole memory argument hinges on counting elements cached *per token, per layer*.

Cached elements per token per layer

MHA caches K and V across all heads:

$$2 \times H_q \times d_{\text{head}} = 2 \times 128 \times 128 = 32,768 \text{ elements} \Rightarrow 65,536 \text{ B (FP16)}.$$

MLA caches only the latent plus the decoupled-RoPE dim:

$$c_{\text{dim}} + \text{rope} = 512 + 64 = 576 \text{ elements} \Rightarrow 1,152 \text{ B (FP16)}.$$

4 Step 3 — The compression ratio

The ratio of MLA's cache to MHA's is just the ratio of those element counts.

MLA vs MHA KV ratio

$$\text{ratio} = \frac{c_{\text{dim}} + \text{rope}}{2 \times H_q \times d_{\text{head}}}.$$

DeepSeek-V3: $c_{\text{dim}}=512$, $\text{rope}=64$, $H_q=128$, $d=128$

$$\text{ratio} = \frac{512 + 64}{2 \times 128 \times 128} = \frac{576}{32,768} = 0.017578 \dots \approx \mathbf{0.0176}.$$

So MLA stores 1.76% of what MHA would, a reduction of

$$1 - \text{ratio} = 1 - 0.0176 = \mathbf{0.9824} = 98.2\%. \checkmark$$

The headline

DeepSeek-V3's MLA caches just 576 numbers per token per layer where full MHA would cache 32,768 — a ratio of 0.0176, i.e. a **98.2%** KV-cache reduction. On a like-for-like 80-layer accounting the per-request cache shrinks roughly $57\times$ (e.g. 32.5 GB $\rightarrow$ 0.57 GB at $T = 6200$). This is the single largest architectural lever for KV memory — far bigger than any GQA grouping or KV quantization, and it is what makes 128K context practical on one node.

5 Step 4 — Why the up-projections are nearly free: the absorbed matmul

A naive reading says “MLA adds two extra matmuls (W^{UK}, W^{UV}) per step — that must be slow.” It is not, because the up-projection can be *absorbed* into the query side.

Absorbing W^{UK} into Q

The attention score with reconstructed $K = c_{KV} W^{UK}$ is

$$s = Q K^\top = Q (c_{KV} W^{UK})^\top = Q (W^{UK})^\top c_{KV}^\top = \underbrace{(Q (W^{UK})^\top)}_{Q'} c_{KV}^\top.$$

We can precompute $Q' = Q (W^{UK})^\top$ once (it does not depend on the cached token), so decode never reconstructs K at all — it does a single small matmul $Q' c_{KV}^\top$ *directly against the cached latent*.

Intuition

The score becomes a dot product against the 512-dim latent instead of against the full reconstructed K . Because the latent is *shared* across all 128 query heads, the model reads one small cached vector per token rather than 128 separate K vectors — so MLA

decode is both smaller in memory *and* competitive in compute with MHA, despite “adding” projections. The matmul moves to the query side, where it costs nothing per cached token.

6 Step 5 — The catch: MLA must be trained in

MLA’s reduction comes with a real constraint.

Watch out

The down- and up-projection matrices (W^{DKV}, W^{UK}, W^{UV}) and the decoupled-RoPE scheme are learned *end-to-end with the model*. You cannot retrofit MLA onto an existing MHA or GQA checkpoint — there is no latent to compress into, and the rotary handling differs. MLA is an architecture decision made before training, not a serving toggle. This is why MLA appears in *new* model families (DeepSeek-V2/V3) rather than as a conversion of older ones, and why GQA — which any model can be designed with cheaply — remains the broader default. For a 236B-class MoE at long context, MLA is the difference between 128K context fitting on one node and not fitting at all.

7 The argument, at a glance

#	Step	Result
1	Idea	cache a shared low-rank latent, reconstruct heads on the fly
2	Forward	down-project $x \rightarrow c_{KV}$; up-project $c_{KV} \rightarrow K, V$
3	Cached	MLA 576 vs MHA 32,768 elements/token/layer
4	Ratio	$(512+64)/(2 \cdot 128 \cdot 128) = 0.0176 \Rightarrow 98.2\%$ less ✓
5	Absorbed	$s = (QW^{UK\top})c_{KV}^\top$: matmul moves to query side
6	Catch	must be trained in; cannot retrofit onto MHA/GQA

7.1 Equation cheat-sheet

Cache (down-proj)	$c_{KV} = x W^{DKV} \quad [1 \times c_{\text{dim}}]$
Reconstruct	$K = c_{KV} W^{UK}, \quad V = c_{KV} W^{UV}$
Ratio vs MHA	$(c_{\text{dim}} + \text{rope}) / (2 H_q d_{\text{head}})$
Reduction	$1 - \text{ratio}$
Absorbed score	$s = (Q W^{UK\top}) c_{KV}^\top$

Appendix A — Reference implementation

Every number above is produced by `m03_t04_mla_engine.py`. Change c_{dim} , the rope dimension, H_q , or d_{head} to size MLA for any model.

```

1 """
2 Reference implementation for: Module 03 - Topic 4
3 "MLA: DeepSeek's Low-Rank KV Compression, Worked Out by Hand."
4
5 MLA caches a small per-token latent (c_dim) plus a decoupled-RoPE dim, instead of
6 full per-head K and V. Ratio vs MHA = (c_dim + rope_dim) / (2 * H_q * d_head).
7
8 Convention: 1 GB = 1e9 bytes, 1 MB = 1e6 bytes (decimal).
```

```

9 Every number printed here is transcribed (rounded) into the worked-by-hand PDF.
10 """
11
12 # -----
13 # 0. DeepSeek-V3 MLA configuration.
14 # -----
15 c_dim    = 512      # compressed KV latent dimension (what we cache)
16 rope_dim = 64       # decoupled-RoPE dimension (also cached, per token)
17 H_q      = 128      # number of query heads
18 d_head   = 128      # head dimension
19 dtype_b  = 2        # FP16, bytes per element
20
21 print("== 0. DeepSeek-V3 MLA config ==")
22 print(f"c_dim={c_dim} rope_dim={rope_dim} H_q={H_q} d_head={d_head} dtype={dtype_b}B")
23
24 # -----
25 # 1. What MHA would cache per token per layer, vs what MLA caches.
26 #   MHA: K and V, each H_q * d_head -> 2 * H_q * d_head elements.
27 #   MLA: the latent c_dim plus the decoupled-rope rope_dim -> c_dim + rope_dim.
28 # -----
29 mha_elems = 2 * H_q * d_head          # K + V across all heads
30 mla_elems = c_dim + rope_dim          # cached latent + decoupled rope
31 print("\n== 1. Cached elements per token per layer ==")
32 print(f"MHA: 2 * H_q * d_head = 2 * {H_q} * {d_head} = {mha_elems} elements")
33 print(f"MLA: c_dim + rope_dim = {c_dim} + {rope_dim} = {mla_elems} elements")
34 print(f"MHA bytes/token/layer = {mha_elems*dtype_b} B")
35 print(f"MLA bytes/token/layer = {mla_elems*dtype_b} B")
36
37 # -----
38 # 2. The compression ratio vs MHA.
39 # -----
40 ratio = (c_dim + rope_dim) / (2 * H_q * d_head)
41 print("\n== 2. MLA vs MHA ratio = (c_dim + rope_dim) / (2 * H_q * d_head) ==")
42 print(f"ratio = ({c_dim} + {rope_dim}) / (2 * {H_q} * {d_head})")
43 print(f"      = {c_dim+rope_dim} / {2*H_q*d_head}")
44 print(f"      = {ratio:.6f}")
45 print(f"reduction vs MHA = 1 - ratio = {(1-ratio)*100:.2f}%")
46 print(f"MLA stores {ratio*100:.2f}% of what MHA would.")
47
48 # -----
49 # 3. Full-cache comparison at a context length (use Nexus-style T=6200,
50 #    and DeepSeek-style long context T=128000), 80 layers for a like-for-like.
51 # -----
52 def cache_bytes(elems_per_tok_layer, L, T):
53     return elems_per_tok_layer * dtype_b * L * T
54
55 print("\n== 3. Full KV cache (illustrative, L=80 layers) ==")
56 for T in (6200, 128000):
57     mha_b = cache_bytes(mha_elems, 80, T)
58     mla_b = cache_bytes(mla_elems, 80, T)
59     print(f"  T={T:>6}: MHA = {mha_b/1e9:8.2f} GB   MLA = {mla_b/1e9:7.3f} GB   "
60           f"({mha_b/mla_b:.1f}x less)")
61
62 # -----
63 # 4. The "absorbed matmul": decode does Q' @ c_KV^T against the small latent
64 #    instead of Q @ K^T against full K. Relative work per score.
65 # -----
66 print("\n== 4. Absorbed-matmul compute (per head, per cached token) ==")
67 mha_score_macs = d_head          # Q[d] . K[d]
68 mla_score_macs = c_dim           # Q'[c_dim] . c_KV[c_dim]
69 print(f"MHA score: Q[{d_head}] . K[{d_head}] -> {mha_score_macs} MACs vs full K")
70 print(f"MLA score: Q'[{c_dim}] . c_KV[{c_dim}] -> {mla_score_macs} MACs vs latent")
71 # Note: per-head MLA latent matmul is larger in raw MACs (512>128) but the latent is

```

```
72 # SHARED across all H_q heads, so per shared cached vector it is far cheaper overall;  
73 # the document explains the memory win, which is the dominant decode bottleneck.  
74  
75 # -----  
76 # 5. Headline numbers used in the document.  
77 # -----  
78 print("\n== 5. Headline ==")  
79 print(f"DeepSeek-V3 MLA ratio = {ratio:.4f} (~{ratio*100:.2f}% of MHA)")  
80 print(f"> ~{(1-ratio)*100:.1f}% KV-cache reduction vs full MHA")
```

— END OF DOCUMENT —